Inventory as an Update Monad: Constant-Position Directed Containers

Extracted from a machine-checked development

Abstract

Directed containers whose position sets are constant are exactly monoid actions. We record the equivalence in both directions, embed the constant-position presentation into the general theory of directed containers, and cointerpret the resulting structure as an *update monad* $TX = S \to M \times X$ whose Kleisli maps observe a stock level and emit a movement. Two supply-chain remarks are machine-checked: abelianization of movement words forgets audit-trail order, and truncated subtraction on $\mathbb{N}$ clamps unmet demand (repaired by passing to $\mathbb{Z}$). A two-product warehouse with an order-up-to policy illustrates the Kleisli reading. The proofs of all stated results have been machine-checked; we record only the mathematical content.

Contents

1 Introduction **1**

2 Monoid actions **2**

3 Constant-position directed containers **2**

4 The equivalence **3**

5 Embedding into directed containers **3**

6 The update monad **4**

7 Two remarks on movements and stock **4**

8 Example: a two-product warehouse **5**

9 Discussion **5**

1 Introduction

Inventory management tracks a stock level and a monoid of stock movements. Applying a movement updates the stock; composing movements associates; the empty movement does nothing. This is a right monoid action of movements on stock levels.

The same data may be read as a directed container whose positions do not depend on the shape: the single position set is the monoid of movements, the root is the unit, and the shift is multiplication. Conversely, every constant-position directed container is a monoid action. Under this correspondence the cointerpretation of Ahman and Uustalu becomes the *update monad*

$$TX \;=\; S \longrightarrow M \times X,$$

whose Kleisli arrows observe a state and emit an update together with a result—precisely the form of an inventory policy that looks at current stock and decides what to order.

What this note claims. In ordinary mathematical language:

1. monoid actions $\longleftrightarrow$ constant-position directed containers, with both round trips definitional;
2. the embedding of the constant-position case into general directed containers is transport-free once one uses that transport along a shape equality is the identity when positions are constant;
3. the update monad of a monoid action satisfies the three monad laws;
4. abelianization of free movement words forgets order (audit trail versus balance), and monus on $\mathbb{N}$ clamps backorders while $\mathbb{Z}$ does not.

We work throughout in a single universe of types. Proofs are indicated by the phrase "by lean," meaning they have been checked by a proof assistant rather than written out in prose.

2 Monoid actions

Definition 2.1 (Monoid)**.** A **monoid** M consists of a type $|M|$, a multiplication $\cdot : |M| \times |M| \to |M|$, and a unit $1 \in |M|$, satisfying associativity and the two unit laws.

Definition 2.2 (Monoid action)**.** A **right monoid action** consists of a monoid M, a type S of states, and a map

$$\triangleleft \;:\; S \times |M| \;\longrightarrow\; S$$

satisfying

$$s \triangleleft 1 \;=\; s, \qquad (s \triangleleft m) \triangleleft m' \;=\; s \triangleleft (m \cdot m'). \tag{1}$$

For inventory, S is the set of stock levels and M the monoid of stock movements; $s \triangleleft m$ applies the movements m to the stock s.

3 Constant-position directed containers

Definition 3.1 (Constant-position directed container)**.** A **constant-position directed container** consists of

1. a type S of shapes (states),
2. a single type M of positions,
3. a sub-shape map $\mathsf{sub} : S \to M \to S$,
4. a root position $r \in M$ (independent of shape),
5. a shift operation $\mathsf{shift} : M \to M \to M$ (independent of shape),

subject to the five laws, stated without transport:

$$\mathsf{sub}\, s\, r \;=\; s, \tag{2}$$

$$\mathsf{shift}\, r\, m \;=\; m, \tag{3}$$

$$\mathsf{shift}\, m\, r \;=\; m, \tag{4}$$

$$\mathsf{sub}\, s\, (\mathsf{shift}\, p\, q) \;=\; \mathsf{sub}\, (\mathsf{sub}\, s\, p)\, q, \tag{5}$$

$$\mathsf{shift}\, (\mathsf{shift}\, p\, q)\, r' \;=\; \mathsf{shift}\, p\, (\mathsf{shift}\, q\, r'). \tag{6}$$

Remark 3.2 (Why constant positions matter)**.** With constant positions, the shift is a genuine binary operation on the single set M. In the general directed-container case, positions vary with shape, so shift is only a fibrewise composition and no global monoid exists on positions. The constant-position case is the sound special case of that observation.

Remark 3.3. We deliberately do *not* formulate the structure as a general directed container plus a type equality $\forall s.\ \mathsf{Pos}(s) = M$. That formulation buries every law in transports. The presentation above keeps the equalities homogeneous.

4 The equivalence

Definition 4.1 (From constant-position DC to action)**.** Given a constant-position directed container C, define a monoid action by taking $|M| = C.M$, multiplication $= C.\mathsf{shift}$, unit $= C.r$, states $= C.S$, and action $= C.\mathsf{sub}$. Laws (3), (4), (6) are the monoid laws; laws (2), (5) are the action laws.

Definition 4.2 (From action to constant-position DC)**.** Given a monoid action of M on S, define a constant-position directed container by taking shapes S, positions $|M|$, sub-shape $=$ the action, root $=$ the monoid unit, and shift $=$ monoid multiplication.

Theorem 4.3 (Round trips)**.** *The constructions of Definitions 4.1 and 4.2 are mutually inverse:*

$$A \longmapsto const \longmapsto action \;=\; A, \qquad C \longmapsto action \longmapsto const \;=\; C. \tag{7}$$

Proof. By lean. □

Remark 4.4. The equalities are definitional: both directions are dictionaries renaming the same data, and the round trips hold by structure η and proof irrelevance.

5 Embedding into directed containers

Lemma 5.1 (Transport is trivial)**.** *Let S and M be types, and let $h : s = t$ be an equality of shapes. For every $q \in M$,*

$$\mathsf{transport}_h(q) \;=\; q. \tag{8}$$

That is, transport of a position along a shape equality is the identity when the position family is constant.

Proof. By lean. □

Definition 5.2 (Embedding)**.** Every constant-position directed container C determines a directed container with

1. shapes $C.S$,
2. $\mathsf{Pos}(s) = C.M$ for every shape s,
3. root constantly equal to $C.r$,
4. sub-shape and shift as in C.

Theorem 5.3 (Laws match)**.** *Under the embedding of Definition 5.2, the five directed-container axioms are exactly the five constant-position laws of Definition 3.1, with the D2/D5 transports erased by Lemma 5.1.*

Proof. By lean. □

Corollary 5.4 (Monoid actions as directed containers)**.** *Every monoid action determines a directed container, by composing Definition 4.2 with Definition 5.2. Directed containers whose position sets are constant are exactly monoid actions.*

Proof. By lean. □

6 The update monad

Definition 6.1 (Update monad)**.** Let A be a monoid action of monoid M on states S. The **update monad** of A is the endofunctor

$$\mathsf{Update}_A X \;=\; S \longrightarrow |M| \times X. \tag{9}$$

A Kleisli map $X \to \mathsf{Update}_A Y$ observes a state in S and emits an update in $|M|$ together with a result in Y.

Definition 6.2 (Unit and bind)**.**

1. $\mathsf{pure} : X \to \mathsf{Update}_A X$ emits the monoid unit and returns the value:
$$\mathsf{pure}(x)(s) \;=\; (1,\, x).$$

2. Given $t : \mathsf{Update}_A X$ and $k : X \to \mathsf{Update}_A Y$,
$$\mathsf{bind}(t, k)(s) \;=\; (m \cdot m',\; y),$$
where $(m, x) = t(s)$, $(m', y) = k(x)(s \triangleleft m)$: run t, apply its update to the state, run k on the updated state, and multiply the two updates.

Theorem 6.3 (Monad laws)**.** *For every monoid action A and types X, Y, Z:*

1. ***Left unit:*** $\mathsf{bind}(\mathsf{pure}(x),\, k) = k(x)$*;*
2. ***Right unit:*** $\mathsf{bind}(t,\, \mathsf{pure}) = t$*;*
3. ***Associativity:*** $\mathsf{bind}(\mathsf{bind}(t, k),\, h) = \mathsf{bind}\big(t,\, x \mapsto \mathsf{bind}(k(x), h)\big)$.

Proof. By lean. □

Remark 6.4. The left unit uses the action unit law (the state is unchanged) and the monoid left unit law. The right unit uses the monoid right unit law. Associativity uses the action multiplication law (the middle state is the same under either bracketing) and monoid associativity.

Remark 6.5 (Cointerpretation)**.** The update monad is the cointerpretation of the constant-position directed container associated to A, in the sense of Ahman and Uustalu: directed containers cointerpret as monads, and the constant-position case specialises to update monads.

7 Two remarks on movements and stock

Definition 7.1 (Movement words)**.** A **stock movement** for a single product is either a receipt of quantity $q \in \mathbb{N}$ or an issue of quantity q. Words in the free monoid on movements are the storekeeper's **audit trail**. The **net effect** of a word is its image in the abelianization $\mathbb{Z}$ (the ledger **balance**):

$$\mathsf{net}([]) = 0, \quad \mathsf{net}(\mathsf{receipt}\, q :: w) = q + \mathsf{net}(w), \quad \mathsf{net}(\mathsf{issue}\, q :: w) = \mathsf{net}(w) - q. \tag{10}$$

Proposition 7.2 (Abelianization forgets order)**.** *Receive*-2-*then-issue*-1 *and issue*-1-*then-receive*-2 *have the same net balance but are distinct as lists:*

$$\mathsf{net}([\mathsf{receipt}\,2,\ \mathsf{issue}\,1]) \ = \ \mathsf{net}([\mathsf{issue}\,1,\ \mathsf{receipt}\,2]), \tag{11}$$

while the two words are unequal. Operationally the difference matters: from a stock of zero the second word passes through a shortage the first never sees.

Proof. By lean. □

Proposition 7.3 (Monus clamps; integers admit backorders)**.** *1. There exist* $s, m \in \mathbb{N}$ *with* $(s \dot{-} m) + m \neq s$ *(truncated subtraction silently discards unmet demand).*

2. For all $s, m \in \mathbb{Z}$*,* $(s - m) + m = s$ *(negative stock is a backorder; every issue is exactly undone by the matching receipt).*

Proof. By lean. □

8 Example: a two-product warehouse

Example 8.1 (Inventory action)**.** Let $M = \mathbb{Z}\times\mathbb{Z}$ under componentwise addition (net movements for two products; the abelianization $\mathbb{Z}^{\mathrm{Prod}}$ with $|\mathrm{Prod}| = 2$), and let $S = \mathbb{Z} \times \mathbb{Z}$ (stock levels, backorders admitted). The action is componentwise addition:

$$(s_1, s_2) \triangleleft (m_1, m_2) \ = \ (s_1 + m_1,\ s_2 + m_2). \tag{12}$$

The associated directed container has stock levels as shapes and movements as positions: a receipt followed by an equal issue is a nontrivial endomorphism of the stock state.

Definition 8.2 (Order-up-to policy)**.** The Kleisli map $\mathsf{policy} : 1 \to \mathsf{Update}_{\mathrm{inv}}1$ is the replenishment rule: for each product, if stock is below the reorder point 5, order up to 10; otherwise order nothing. Goods inward $\mathsf{receive}(m)$ is the Kleisli map that unconditionally emits the movement m.

Proposition 8.3 (Policy behaviour)**.** *1. At stock* $(2, 7)$*, the policy emits movement* $(8, 0)$*.*

2. Applying that movement yields stock $(10, 7)$*.*

3. Kleisli composition threads state: receiving $(3, 0)$ *before the policy, starting from stock* $(1, 7)$*, produces total movement* $(9, 0)$ *(the policy sees stock* 4 *and orders* 6*).*

Proof. By lean. □

9 Discussion

What has been established. Constant-position directed containers and monoid actions are the same data. Their cointerpretation is the update monad, with the three monad laws following from the monoid and action laws. The two remarks separate the free monoid of audit trails from its abelianization (balances), and motivate working over $\mathbb{Z}$ rather than $\mathbb{N}$ when unmet demand must be recorded.

Scope. The development treats a single universe of types and does not address multi-echelon inventory, stochastic demand, or cost optimisation. The warehouse example is a pure illustration of the Kleisli reading.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

Acknowledgements

The mathematical content follows the cointerpretation of directed containers as monads due to Ahman and Uustalu, specialised to the constant-position case and read as inventory. The presentation here is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] D. Ahman and T. Uustalu. Directed containers as categories. In *MSFP*, volume 207 of *EPTCS*, pages 89–98, 2016.

[2] D. Ahman and T. Uustalu. Update monads: cointerpreting directed containers. In *TLCA*, volume 8560 of *LNCS*, pages 1–15. Springer, 2014.

[3] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.